

BÁO CÁO THỰC HÀNH LẬP TRÌNH MẠNG BUỔI 5

- **Họ và tên:** Phan Hoàng Hải
 - **Mã số sinh viên:** 20225715
-

I. Kịch bản 1: Echo cơ bản

- **Thao tác thực hiện:** Chạy 1 server và 1 client trên cùng máy (localhost). Client gửi 3 chuỗi ký tự khác nhau và chờ server phản hồi.
 - **Kết quả ghi nhận:** Client nhận lại chính xác 3 chuỗi đã gửi. Server ghi log đã nhận và phản hồi thành công cho cùng một địa chỉ client.

```
pypy@HN-LA-HaiPH16:/mnt/d/NetworkProgramming/Week04_InClass$ ./server
Server running...waiting for connections.
Received request...
String received from and resent to the client:hello world from haiph

String received from and resent to the client:xin chao the gioi
aiph

Received request...
String received from and resent to the client:hello world
gioi
aiph

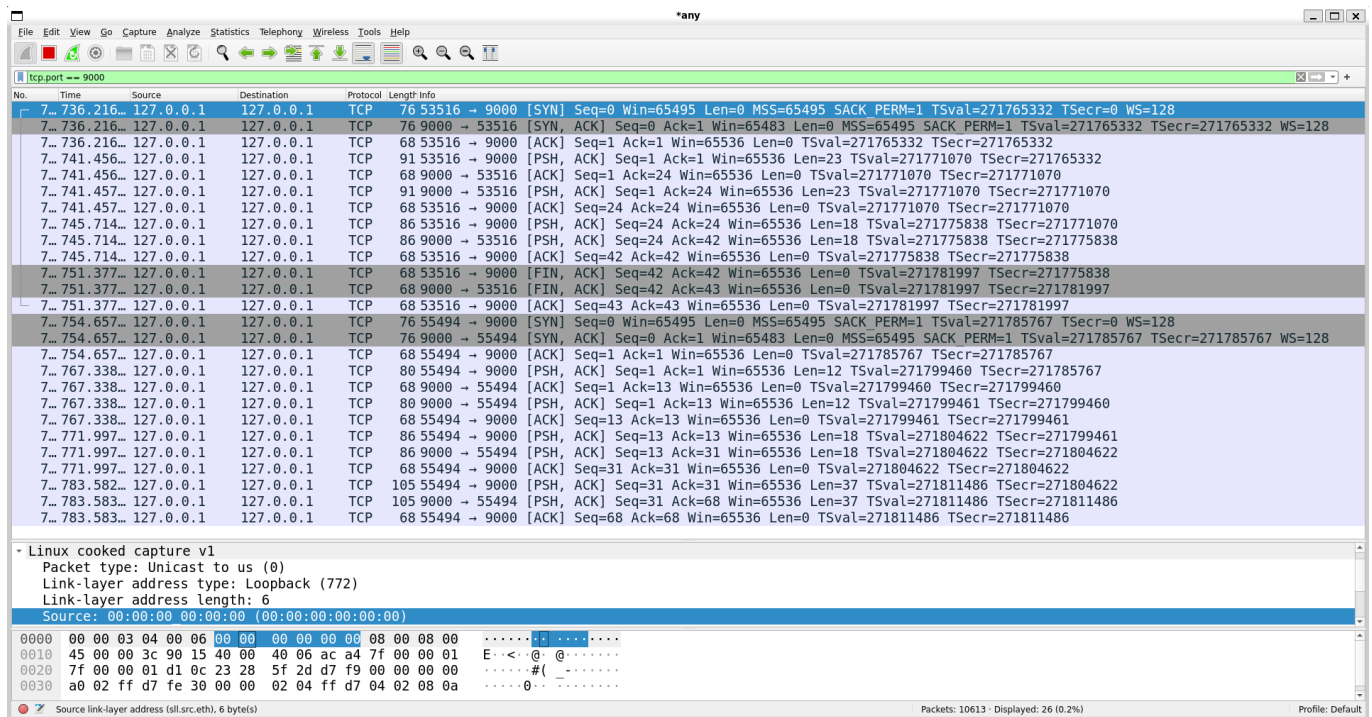
String received from and resent to the client:xin chao the gioi
aiph

String received from and resent to the client:hello13u432498324$$$#@ 485489594385
```

Log trên 3 Clients:

```
pypy@HN-LA-HaiPH16:/mnt/d/NetworkProgramming/Week04_InClass$ ./client 127.0.0.1
hello world
String received from the server: hello world
xin chao the gioi
String received from the server: xin chao the gioi
hello13u432498324$$$#@ 485489594385
String received from the server: hello13u432498324$$$#@ 485489594385
```

Wireshark TCP Port 9000:



- Giải thích:** Client và server thiết lập kết nối TCP bằng bắt tay 3 bước (SYN, SYN-ACK, ACK). Client (53516) gửi mỗi chuỗi ký tự bằng lệnh send(), nên stack TCP của client gửi các phân đoạn dữ liệu kèm cờ PSH,ACK để yêu cầu bên nhận đẩy dữ liệu lên ứng dụng ngay lập tức và đồng thời xác nhận thứ tự (ACK). Server nhận qua recv(), in log và gọi send() để trả lại cùng dữ liệu về client (destination port 53516), do đó stack TCP của server cũng gửi các phân đoạn với PSH,ACK. Vì vậy trong Wireshark sẽ thấy các gói PSH,ACK theo cả hai chiều (PSH là gợi ý đẩy dữ liệu lên ứng dụng, ACK là xác nhận nhận dữ liệu). Vòng lặp recv()/send() trong server đảm bảo mỗi lần nhận được dữ liệu thì sẽ echo trả ngay cho client.

II. Kịch bản 2: Nhiều client đồng thời

- Mục tiêu:** Quan sát cách server TCP xử lý yêu cầu từ nhiều client. Log server:

```
pypy@HN-LA-HaiPH16:/mnt/d/NetworkProgramming/Week04_InClass$ ./server
Server running...waiting for connections.
Received request...
String received from and resent to the client:xin chao the gioi

String received from and resent to the client:tiep tục
the gioi
```

- Kết quả ghi nhận:** Log trên server chỉ hiển thị các thông điệp từ client được server chấp nhận và xử lý đầu tiên.

```

py@HN-LA-HaiPH16:/mnt/d/NetworkProgramming/Week04_InClass$ ./client 127.0.0.1
xin chao the gioi
String received from the server: xin chao the gioi
tiap tuc
String received from the server: tiap tuc
the gioi
[ ]

py@HN-LA-HaiPH16:/mnt/d/NetworkProgramming/Week04_InClass$ ./client 127.0.0.1
hello world
[ ]

py@HN-LA-HaiPH16:/mnt/d/NetworkProgramming/Week04_InClass$ ./client 127.0.0.1
hee
[ ]

```

Wireshark TCP Port 9000:

3...	4785.93...	127.0.0.1	127.0.0.1	TCP	76 37930 → 9000	[SYN]	Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=276200510 TSecr=0 WS=128
3...	4785.93...	127.0.0.1	127.0.0.1	TCP	76 9000 → 37930	[SYN, ACK]	Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=276200510 TSecr=276200510 WS=128
3...	4785.93...	127.0.0.1	127.0.0.1	TCP	68 37930 → 9000	[ACK]	Seq=1 Ack=1 Win=65536 Len=0 TSval=276200510 TSecr=276200510
4...	4857.80...	127.0.0.1	127.0.0.1	TCP	76 55026 → 9000	[SYN]	Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=276284840 TSecr=0 WS=128
4...	4857.80...	127.0.0.1	127.0.0.1	TCP	76 9000 → 55026	[SYN, ACK]	Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=276284840 TSecr=276284840 WS=128
4...	4857.80...	127.0.0.1	127.0.0.1	TCP	68 55026 → 9000	[ACK]	Seq=1 Ack=1 Win=65536 Len=0 TSval=276284840 TSecr=276284840
4...	4862.24...	127.0.0.1	127.0.0.1	TCP	76 35160 → 9000	[SYN]	Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM=1 TSval=276289749 TSecr=0 WS=128
4...	4862.24...	127.0.0.1	127.0.0.1	TCP	76 9000 → 35160	[SYN, ACK]	Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM=1 TSval=276289749 TSecr=276289749 WS=128
4...	5017.28...	127.0.0.1	127.0.0.1	TCP	68 35160 → 9000	[ACK]	Seq=1 Ack=1 Win=65536 Len=0 TSval=276289749 TSecr=276289749
4...	5017.28...	127.0.0.1	127.0.0.1	TCP	86 37930 → 9000	[PSH, ACK]	Seq=1 Ack=1 Win=65536 Len=18 TSval=276452441 TSecr=276200510
4...	5017.28...	127.0.0.1	127.0.0.1	TCP	68 9000 → 37930	[ACK]	Seq=1 Ack=19 Win=65536 Len=0 TSval=276452441 TSecr=276452441
4...	5017.28...	127.0.0.1	127.0.0.1	TCP	86 9000 → 37930	[PSH, ACK]	Seq=1 Ack=19 Win=65536 Len=18 TSval=276452441 TSecr=276452441
4...	5017.28...	127.0.0.1	127.0.0.1	TCP	68 37930 → 9000	[ACK]	Seq=19 Ack=19 Win=65536 Len=0 TSval=276452441 TSecr=276452441
4...	5021.40...	127.0.0.1	127.0.0.1	TCP	80 55026 → 9000	[PSH, ACK]	Seq=1 Ack=1 Win=65536 Len=12 TSval=276456559 TSecr=276284840
4...	5021.40...	127.0.0.1	127.0.0.1	TCP	68 9000 → 55026	[ACK]	Seq=1 Ack=13 Win=65536 Len=0 TSval=276456559 TSecr=276456559
4...	5039.91...	127.0.0.1	127.0.0.1	TCP	72 35160 → 9000	[PSH, ACK]	Seq=1 Ack=1 Win=65536 Len=4 TSval=276475064 TSecr=276289749
4...	5039.91...	127.0.0.1	127.0.0.1	TCP	68 9000 → 35160	[ACK]	Seq=1 Ack=5 Win=65536 Len=0 TSval=276475064 TSecr=276475064
4...	5032.15...	127.0.0.1	127.0.0.1	TCP	77 37930 → 9000	[PSH, ACK]	Seq=19 Ack=19 Win=65536 Len=9 TSval=276478092 TSecr=276452441
4...	5032.15...	127.0.0.1	127.0.0.1	TCP	77 9000 → 37930	[PSH, ACK]	Seq=19 Ack=28 Win=65536 Len=9 TSval=276478093 TSecr=276478092
4...	5032.15...	127.0.0.1	127.0.0.1	TCP	68 37930 → 9000	[ACK]	Seq=28 Ack=28 Win=65536 Len=0 TSval=276478093 TSecr=276478093

Protocol: IPv4 (0x0800)	
0000	00 00 03 04 00 06 00 00 00 00 00 00 0e 5f 08 00
0010	45 00 00 38 05 d5 40 00 40 06 3e e9 7f 00 00 01
0020	7f 00 00 01 89 58 23 28 c3 13 4d 70 76 fa 9c a0
0030	80 18 02 00 fe 2c 00 00 01 01 08 0a 10 7a ac b8
0040	10 77 d8 d5 68 65 65 0a

Giải thích:

- Server đang xử lý client 1 trong vòng lặp recv()/send(), nên không thể gọi accept() để xử lý client 2 và 3. Các client này chỉ tồn tại trong hàng đợi kernel (thấy qua Wireshark), nhưng dữ liệu của chúng không được xử lý cho đến khi client 1 ngắt kết nối.
- Kernel đã thiết lập kết nối TCP cho cả 3 client (SYN, SYN-ACK, ACK), nhưng mã ứng dụng server không đọc dữ liệu từ client 2 và 3 do đang bận với client 1.

III. Kịch bản 3: Nhiều bản tin bị gộp tại server

- Mục tiêu:** Thử nghiệm và quan sát việc server gộp 2 thông điệp gửi liên tiếp.
- Thao tác thực hiện:** send() 2 chuỗi "bull" và "dog" liên tiếp, quan sát.

[illegible][illegible]

- 5 / 5